# The Knight's Tour

Graph Theory Midterm — Task 3

**Group Members:** Denis, Hugo, Melvyn, Mikal | **Task 3:** Mikal | **Instructor:** Ilham Gurat Adilion | October 21, 2026

## 1. Problem & Graph Model

Given an N × M board (`5 5`) and a start square (`2 2`, 0-indexed), print the squares a knight visits so that it lands on **every square exactly once**, or `path not found`. We model the board as the **knight's graph**: each square is a vertex, and an edge joins two squares one knight jump apart ({|Δx|, |Δy|} = {1, 2}), so every vertex has degree 2 to 8. A Knight's Tour is a **Hamiltonian path** of this graph starting at the given vertex, a problem that is **NP-complete** on general graphs.

## 2. Instant "path not found" (parity arguments)

**Colour.** Every jump changes x + y by an odd number, so the knight alternates colours. On a board with an odd number of squares there is one extra white square (x + y even), so a tour must start on white. Example: 5 × 5 from (0, 1) has no tour.

**4 × n.** On a board 4 squares wide, a jump from an outer line (0 or 3) always lands on an inner line (1 or 2). With 2n outer and 2n inner squares, outer squares sit at every other position of the path. A path starting inside would put all outer squares at even positions, so all of the same colour, but the outer lines hold n white and n black squares. So a tour must start on an outer line. We checked this rule against exhaustive search on every square of 4×5 to 4×8.

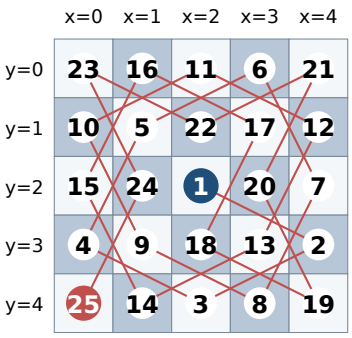
## 3. Algorithm

**Phase 1: Warnsdorff's rule (1823).** Always jump to the unvisited square with the **fewest onward moves**, so corners and borders are visited before they become dead ends. The walk is greedy (no undo), so it either completes the tour or gets stuck. Ties matter on large or narrow boards, so we try 16 deterministic tie-breaks (8 move orders × border-first or centre-first), then 200 random ones with a fixed seed. The first complete walk is printed.

**Phase 2: backtracking with pruning.** If every greedy walk fails, an iterative DFS explores all paths in Warnsdorff order, undoing the last move at each dead end. After every move it backtracks at once if (1) an unvisited square has become unreachable, (2) two squares are forced to be the last square, since a square with one free neighbour can be entered but not left, or (3) the unvisited squares are disconnected (BFS). Because this search is complete, `path not found` is only printed when no tour exists.

## 4. Execution on the Sample (5 × 5, start (2, 2))

| # | From | Candidates (d = free exits) | Pick | Reason |
|---|---|---|---|---|
| 2 | (2,2) | 8 squares, all d2 | (4,3) | All tie (degree and distance): first in move order. |
| 3 | (4,3) | (2,4) d3, (3,1) d3 | (2,4) | Tie: (2,4) is farther from the centre. |
| 4 | (2,4) | (0,3) d1, (1,2) d5, (3,2) d5 | (0,3) | Only one exit left: visit it now or lose it. |
| 6 | (1,1) | (3,2) d4, (2,3) d5, (3,0) d1 | (3,0) | Border square first. |
| 25 | (1,2) | (0,4) d0 | (0,4) | 25 / 25 visited: tour complete. |




*The first greedy walk succeeds (0 restarts, 0 backtracks). Right: visiting order, 1 = start, 25 = end.*

## 5. Complexity (V = N·M squares, E ≤ 4V)

| Part | Time | Why |
|---|---|---|
| Parity checks | O(1) | Arithmetic on N, M and the start. |
| Build graph | O(V) | 8 jumps tested per square. |
| One Warnsdorff walk | O(V) | V steps × ≤ 8 candidates. Degrees are updated in O(1) per visit (≤ 8 neighbours). |
| Phase 1 (216 walks) | O(V) | A constant number of linear walks. |
| Phase 2 | $O(V \cdot 7^V)$ | ≤ 7 children per node, depth V, and an O(V) pruning test per node. Exponential, as expected for an NP-complete problem; only needed in practice to prove 3×3 or 4×4 impossible. |

Memory is O(V). Overall: **O(V)** whenever Phase 1 succeeds, exponential only in the worst case.

## 6. Results & Conclusion

| Board | Start | Result | Time (ms) | Decided by |
|---|---|---|---|---|
| 5 × 5 | (2, 2) | found | 0.4 | Phase 1 |
| 5 × 5 | (0, 1) | path not found | 0.0 | colour |
| 4 × 5 | (1, 0) | path not found | 0.0 | 4 × n |
| 3 × 3 | (0, 0) | path not found | 8.7 | Phase 2 |
| 4 × 4 | (0, 0) | path not found | 18.5 | Phase 2 |
| 8 × 8 | (0, 0) | found | 0.5 | Phase 1 |
| 4 × 11 | (0, 5) | found | 4.4 | 17 restarts |
| 50 × 50 | (10, 7) | found | 31.6 | Phase 1 |
| 150 × 150 | (0, 0) | found | 284.6 | Phase 1 |

Every start square of every board up to 12 × 12 was tested and every tour was checked automatically (all squares once, legal jumps). Time grows linearly with V. **Limitation:** a few very long, narrow boards (e.g. 3 × 40 from some squares) take too long, which is expected for an NP-complete problem.

**Conclusion.** Seen as a Hamiltonian path in the knight's graph, the tour is found in linear time by Warnsdorff's rule on almost every board; backtracking with pruning keeps the answer correct when the rule fails, and the parity arguments answer many impossible inputs instantly. Run: `printf "5 5\n2 2\n" | python3 knights_tour.py` (add `--simulate` to watch the search).

References: H. C. von Warnsdorff, 1823; A. J. Schwenk, *Math. Magazine* 64(5), 1991; I. Parberry, *Discrete Appl. Math.* 73, 1997.